

Data Structure: 정렬 (2)

Prof. Jungmin Kwon
Spring, 2026

■ 학습목표

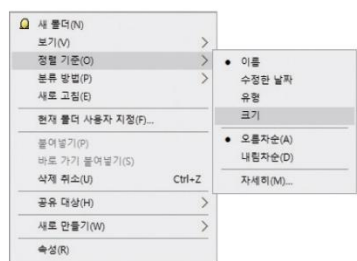
- 자료의 정렬 방법을 알아본다.
- 정렬 방법에 대한 알고리즘을 이해한다.
- 정렬 알고리즘의 효율을 알아본다.

■ 내용

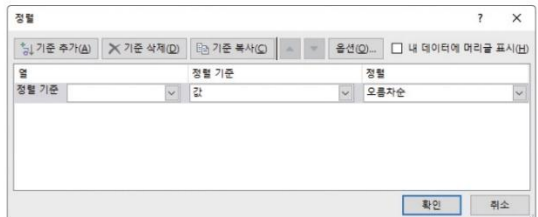
01 정렬의 이해		02 선택 정렬
03 버블 정렬		04 퀵 정렬
05 삽입 정렬		06 셸 정렬
07 병합 정렬		08 기수 정렬
09 힙 정렬		10 트리 정렬

■ 정렬sort

- 순서 없이 배열된 자료를 작은 것부터 큰 것 순서인 오름차순Ascending이나 큰 것부터 작은 것 순서인 내림차순Descending으로 재배열하는 것
- 키(Key) - 자료를 정렬하는데 사용하는 기준이 되는 특정 값
- 정렬의 예



(a) 아이콘 정렬하기



(b) 데이터 시트에서 정렬하는 기준 정하기



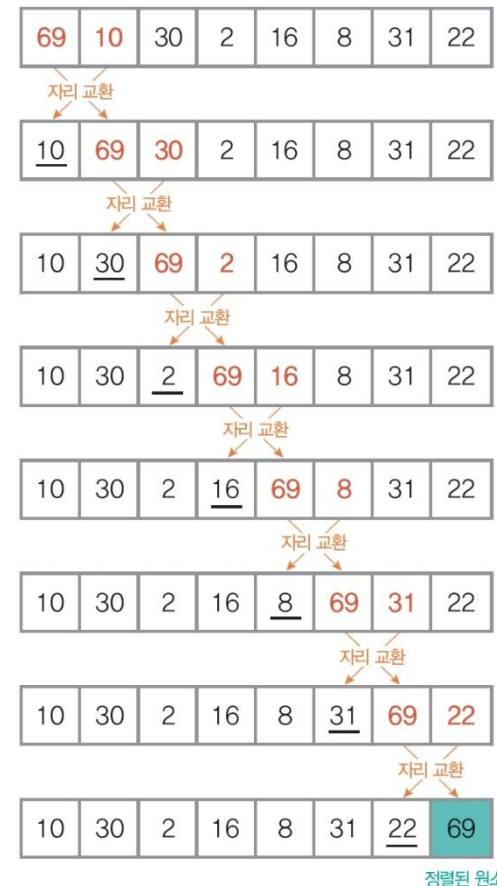
(c) 쇼핑몰에서 제품 정렬하기

그림 9-1 정렬의 예

■ 버블 정렬 bubble sort의 이해

– 인접한 두 개의 원소를 비교하여 자리를 교환하는 방식

- 첫 번째 원소부터 마지막 원소까지 반복하여 한 단계가 끝나면 가장 큰 원소가 마지막 자리로 정렬



■ 삽입 정렬 insert sort의 이해

- 정렬되어있는 부분집합에 정렬할 새로운 원소의 위치를 찾아 삽입하는 방법
- 정렬할 자료를 두 개의 부분집합 $S^{\text{Sorted Subset}}$ 와 $U^{\text{Unsorted Subset}}$ 로 가정
 - 부분집합 S : 정렬된 앞부분의 원소들
 - 부분집합 U : 아직 정렬되지 않은 나머지 원소들

정렬할 원소 : 69 10 30 2 16 8 31 22

《《《《《《《《 삽입 정렬 수행 》》》》》》》》》》

1단계 : 10 69 30 2 16 8 31 22

2단계 : 10 30 69 2 16 8 31 22

3단계 : 2 10 30 69 16 8 31 22

4단계 : 2 10 16 30 69 8 31 22

5단계 : 2 8 10 16 30 69 31 22

6단계 : 2 8 10 16 30 31 69 22

7단계 : 2 8 10 16 22 30 31 69

6. 셸 정렬

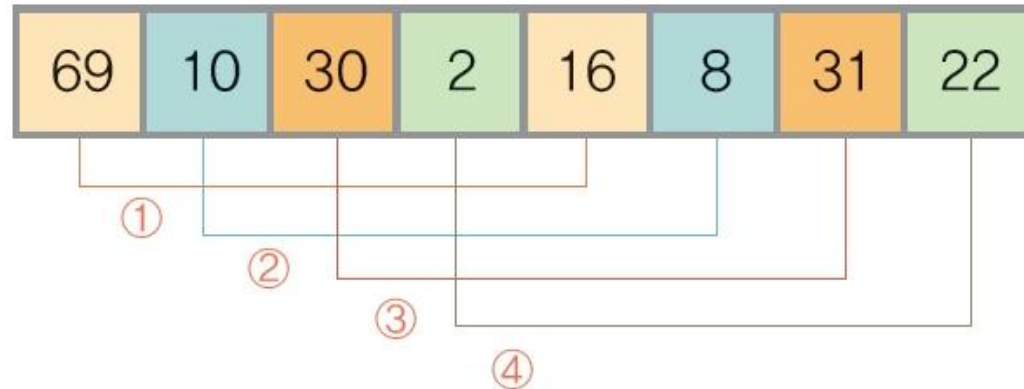
■ 셸 정렬 shell sort의 이해

- 일정한 간격 interval으로 떨어져있는 자료들끼리 부분집합을 구성하고 각 부분집합에 있는 원소들에 대해서 **삽입 정렬**을 수행하는 작업을 반복하면서 전체 원소들을 정렬하는 방법
 - 전체 원소에 대해서 삽입 정렬을 수행하는 것보다 부분집합으로 나누어 정렬하게 되면 비교연산과 교환연산 감소
- 셸 정렬의 부분집합
 - 부분집합의 기준이 되는 간격을 **매개변수 h**에 저장
 - 한 단계가 수행될 때마다 h의 값을 감소시키고 셸 정렬을 순환 호출
 - h가 1이 될 때까지 반복
- 셸 정렬의 성능은 매개변수 h의 값에 따라 달라짐
 - 정렬할 자료의 특성에 따라 매개변수 생성 함수를 사용
 - 일반적으로 사용하는 h의 값은 원소 개수의 $1/2$ 을 사용하고 한 단계 수행될 때마다 h의 값을 반으로 감소시키면서 반복 수행

6. 셸 정렬

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 셸 정렬 방법으로 정렬하는 과정

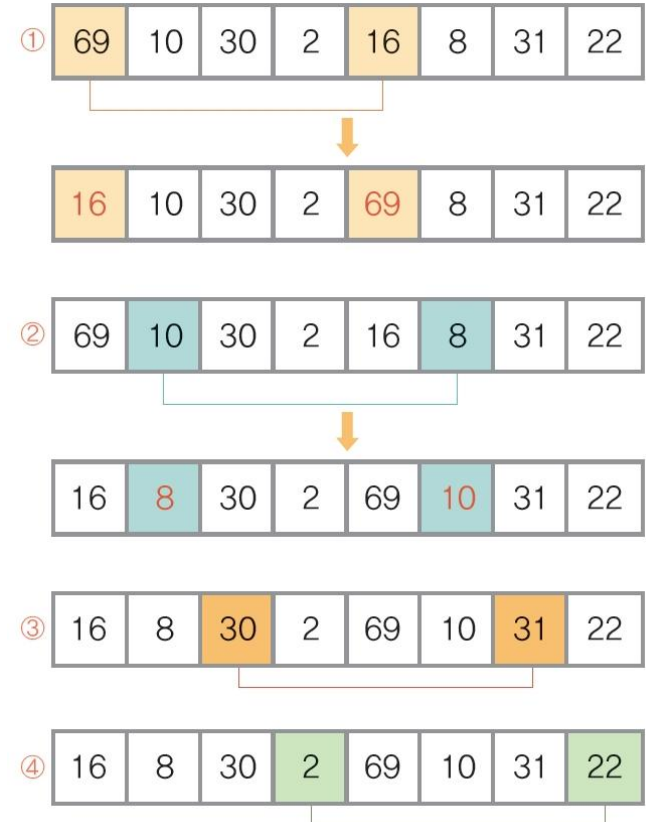
- ① 원소 개수가 여덟 개이므로 매개변수 h 는 4에서 시작한다. $h=4$ 이므로 간격이 4만큼 떨어져 있는 원소들을 같은 부분집합으로 만들면 네 개의 부분집합이 만들어짐(같은 부분집합은 동일한 색으로 표시)



6. 셀 정렬

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 셀 정렬 방법으로 정렬하는 과정

- ① 첫 번째 부분집합 {69, 16}에 대해서 삽입 정렬을 수행하여 정렬
- ② 두 번째 부분집합 {10, 8}에 대해서 삽입 정렬을 수행
- ③ 세 번째 부분집합 {30, 31}에 대해서 삽입 정렬을 수행. $30 < 31$ 이므로 자리 이동은 이루어지지 않음.
- ④ 네 번째 부분집합 {2, 22}에 대해서 삽입 정렬을 수행. $2 < 22$ 이므로 자리 이동은 이루어지지 않음

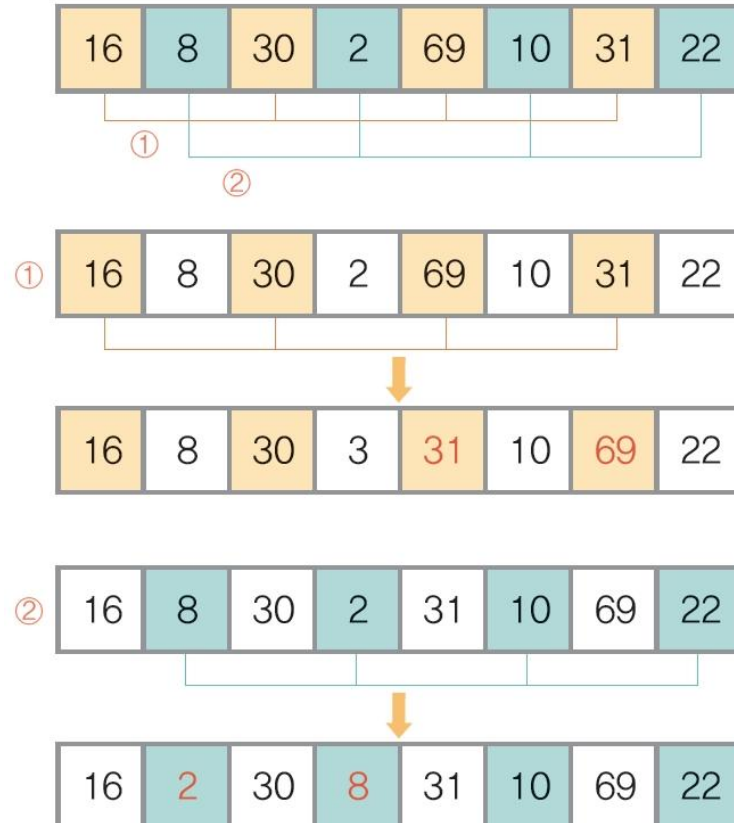


6. 셀 정렬

② 이제 h 를 2로 변경하고 다시 셀 정렬을 시작. $h=2$ 이므로 간격이 2만큼 떨어진 원소들을 같은 부분집합으로 만들면 두 개의 부분집합이 만들어짐

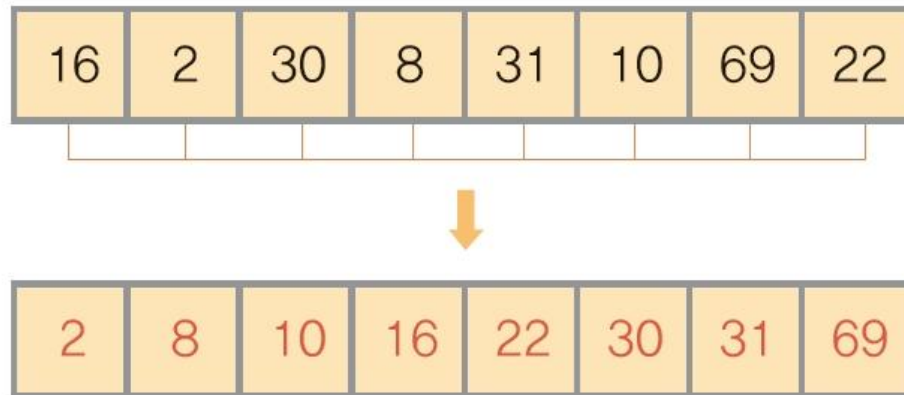
① 첫 번째 부분집합 {16, 30, 69, 31}에 대해 삽입 정렬을 수행하여 정렬

② 두 번째 부분집합 {8, 2, 10, 22}에 대해 삽입 정렬을 수행하여 정렬



6. 셀 정렬

- ③ 이제 h 를 1로 변경하고 다시 셀 정렬을 시작. $h=1$ 이므로 간격이 1만큼 떨어져 있는 원소들을 같은 부분집합으로 만들면 한 개의 부분집합이 만들어짐. 즉 전체 원소에 대해서 삽입 정렬을 수행



■ 셸 정렬 알고리즘

알고리즘 9-6 셸 정렬

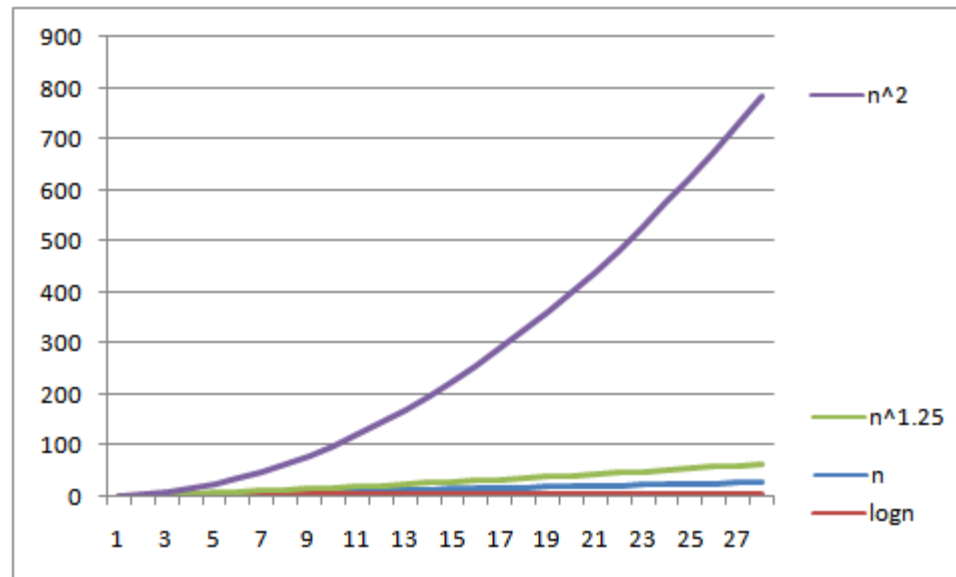
```
shellSort(a[], n)
    interval ← n;
    while (interval ≥ 1) do {
        interval ← interval / 2;
        for (i ← 0; i < interval; i ← i + 1) do {
            intervalSort(a[], i, n, interval);
        }
    }
end shellSort()
```

알고리즘 9-7 셸 부분집합에서의 삽입 정렬

```
intervalSort(a[], begin, end, interval)
    for (i ← begin+interval; i ≤ end; i ← i+interval) do {
        item ← a[i];
        for (j ← i-interval; j ≥ begin and item < a[j]; j ← j-interval) do
            a[j+interval] ← a[j];
        a[j+interval] ← item;
    }
end intervalSort()
```

6. 셀 정렬

- 메모리 사용공간
 - n 개의 원소에 대하여 n 개의 메모리와 매개변수 h 에 대한 저장공간 사용
- 연산 시간
 - 비교횟수
 - 처음 원소의 상태에 상관없이 매개변수 h 에 의해 결정
 - 일반적인 시간 복잡도 : $O(n^{1.25})$
 - 셀 정렬은 삽입 정렬의 시간 복잡도 $O(n^2)$ 보다 개선된 정렬 방법



6. 셸 정렬

- [예제 9-6] 셸 정렬하기 : [교재 544p](#)
- 실행 결과

정렬할 원소 : 69 10 30 2 16 8 31 22

<<<<<<<<< 셸 정렬 수행 >>>>>>>>>>>>

interval=4 >> 16 8 30 2 69 10 31 22

interval=2 >> 16 2 30 8 31 10 69 22

interval=1 >> 2 8 10 16 22 30 31 69

7. 병합 정렬

■ 병합 정렬 merge sort의 이해

- 여러 개의 정렬된 자료의 집합을 병합하여 한 개의 정렬된 집합으로 만드는 방법
- 부분집합으로 분할 divide하고, 각 부분집합에 대해서 정렬 작업을 완성 conquer한 후에 정렬된 부분집합들을 다시 결합 combine하는 분할 정복 divide and conquer 기법 사용
- 병합 정렬 방법의 종류
 - 2-way 병합 : 2개의 정렬된 자료의 집합을 결합하여 하나의 집합으로 만드는 병합 방법
 - n-way 병합 : n개의 정렬된 자료의 집합을 결합하여 하나의 집합으로 만드는 병합 방법

– 2-way 병합 정렬 과정

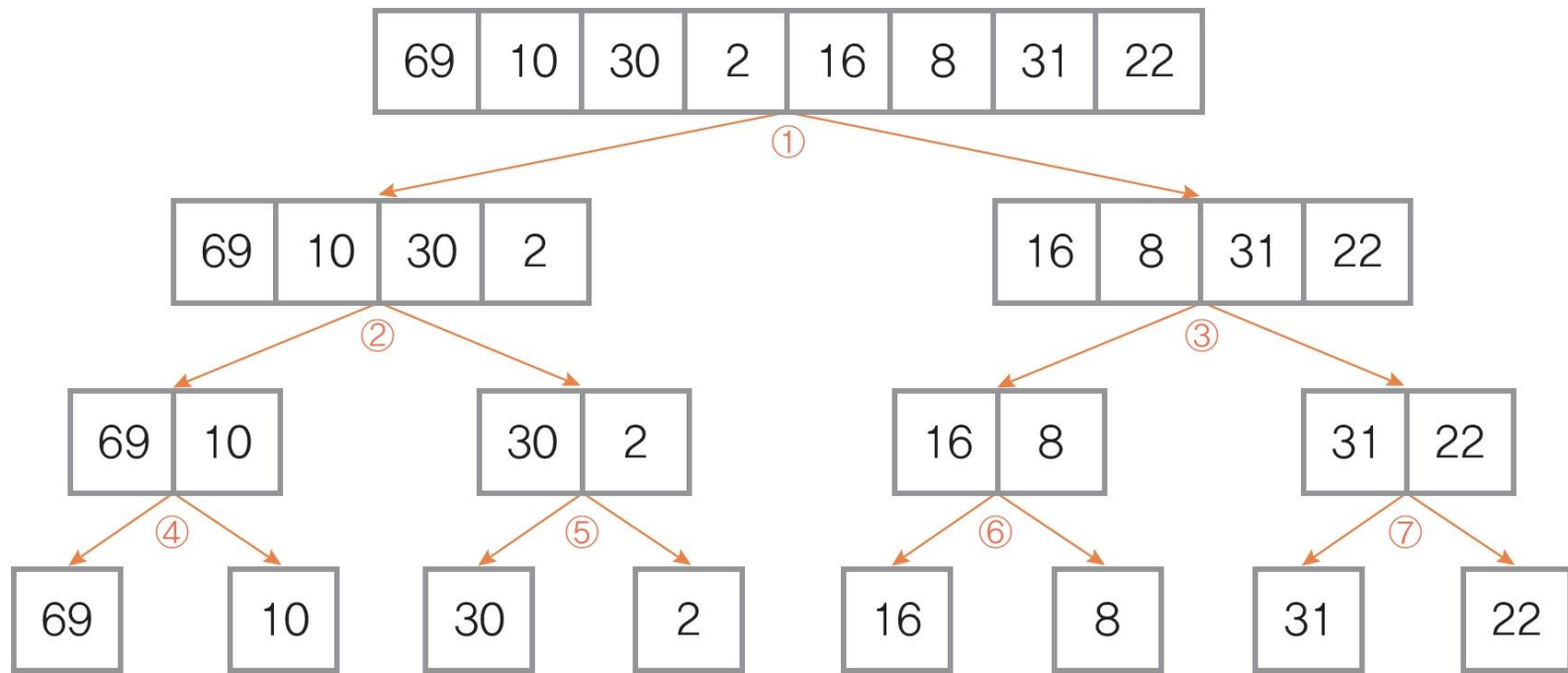
- 분할^{Divide} : 자료들을 두 개의 부분집합으로 분할한다.
- 정복^{Conquer} : 부분집합에 있는 원소를 정렬한다.
- 결합^{Combine} : 정렬된 부분집합들을 하나의 집합으로 정렬하여 결합한다.

그림 9-5 2-way 병합 정렬 과정

7. 병합 정렬

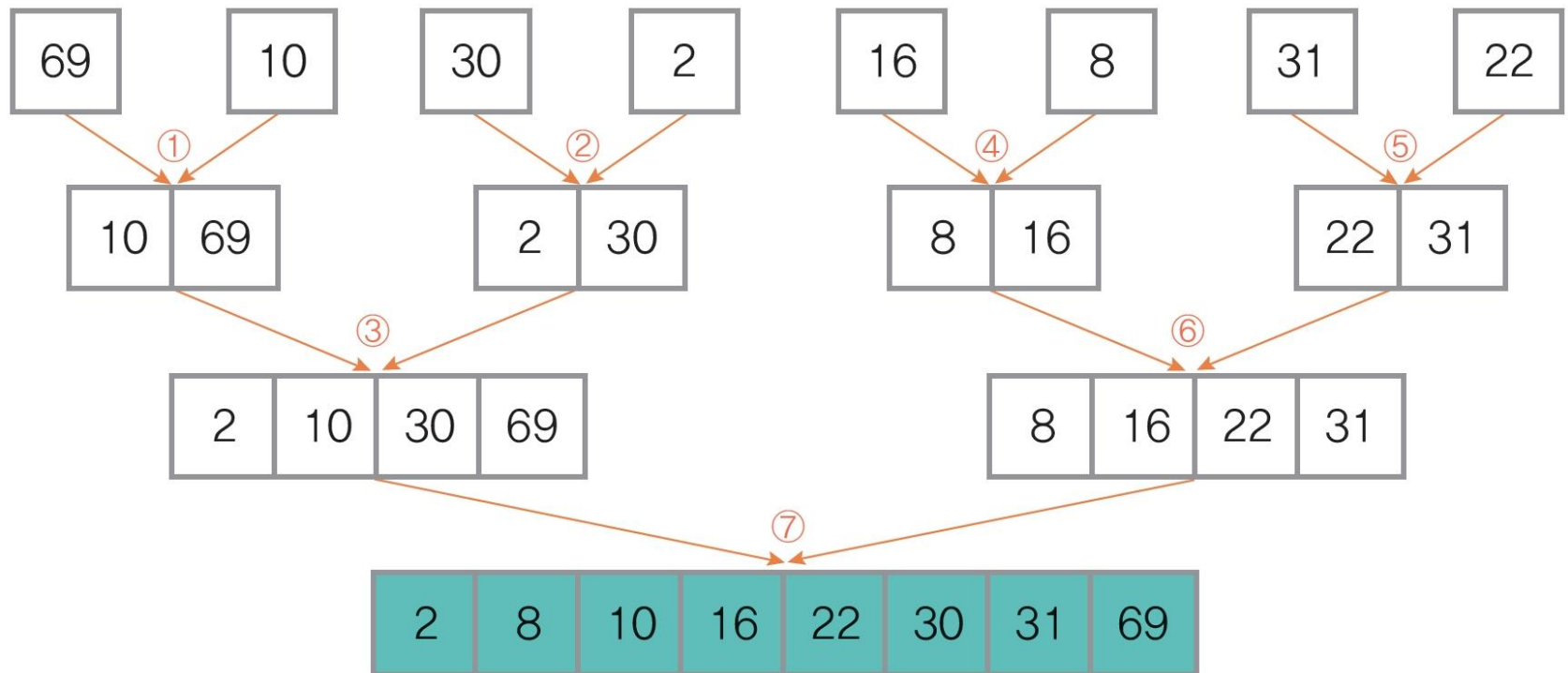
- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 병합 정렬 방법으로 정렬하는 과정

① 분할 단계 : 정렬할 전체 자료의 집합에 대해서 최소 원소의 부분집합이 될 때까지 분할 작업을 반복하여 한 개의 원소를 가진 부분집합 8개 만들



부분집합 여덟 개

- ② 정복과 결합 단계 : 부분집합 두 개를 정렬하여 하나로 결합. 전체 원소가 집합 하나로 묶일 때까지 반복



7. 병합 정렬

■ 병합 정렬 알고리즘

알고리즘 9-8 병합 정렬

```

mergeSort(a[], m, n)
  if (a[m:n]의 원소 수 > 1) then {
    전체 집합을 부분집합 두 개로 분할;
    mergeSort(a[], m, middle);           // 왼쪽 부분집합을 다시 두 개로 분할
    mergeSort(a[], middle + 1, n);       // 오른쪽 부분집합을 다시 두 개로 분할
    merge(a[m:middle], a[middle + 1:n]); // 부분집합 두 개를 하나로 병합
  }
end mergeSort()

```

7. 병합 정렬

알고리즘 9-9 부분집합의 병합

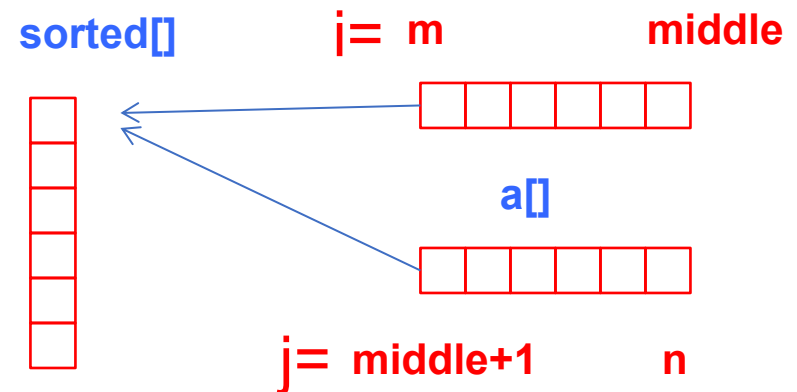
```

merge(a[m:middle], a[middle + 1:n])
  i ← m;           // 첫 번째 부분집합의 첫째 원소 설정
  j ← middle + 1; // 두 번째 부분집합의 첫째 원소 설정
  k ← m;           // 병합 집합의 첫째 원소 설정

  while (i ≤ middle and j ≤ n) do {
    if (a[i] ≤ a[j]) then {
      sorted[k] ← a[i];
      i ← i + 1;
    }
    else {
      sorted[k] ← a[j];
      j ← j + 1;
    }
    k ← k + 1;
  }

  if (i > middle) then 두 번째 부분집합에 남아 있는 원소를 병합 집합 sorted에 복사;
  else 첫 번째 부분집합에 남아 있는 원소를 병합 집합 sorted에 복사;
end merge()
  
```

i, j 가 최대값이 될때까지



– 메모리 사용공간

- 각 단계에서 새로 병합하여 만든 부분집합을 저장할 공간이 추가로 필요
- 원소 n 개에 대해서 $(2 \times n)$ 개의 메모리 공간 사용

– 연산 시간

- 분할 단계 : n 개의 원소를 두개로 분할하기 위해서 $\log_2 n$ 번의 단계 수행
- 병합 단계 : 부분집합의 원소를 비교하면서 병합하는 단계에서 최대 n 번의 비교 연산 수행
- 전체 병합 정렬의 시간 복잡도 : $O(n \log_2 n)$

7. 병합 정렬

- [예제 9-7] 병합 정렬하기 : [교재 548p](#)
- 실행 결과

정렬할 원소 >> 69 10 30 2 16 8 31 22

<<<<<<<< 병합 정렬 수행 >>>>>>>>

병합 정렬 >> 10 69 30 2 16 8 31 22

병합 정렬 >> 10 69 2 30 16 8 31 22

병합 정렬 >> 2 10 30 69 16 8 31 22

병합 정렬 >> 2 10 30 69 8 16 31 22

병합 정렬 >> 2 10 30 69 8 16 22 31

병합 정렬 >> 2 10 30 69 8 16 22 31

병합 정렬 >> 2 8 10 16 22 30 31 69

7. 병합 정렬

예제 9-7 병합 정렬하기

[mergeSort.c]

```

01  #include <stdio.h>
02  #define MAX 30
03  extern int size;
04  int sorted[MAX]; // 원소를 병합하면서 정렬한 상태로 저장할 배열 선언
05
06  void merge(int a[], int m, int middle, int n) {
07      int i, j, k, t;
08      i = m; // 첫 번째 부분집합의 시작 위치 설정
09      j = middle + 1; // 두 번째 부분집합의 시작 위치 설정
10      k = m; // 배열 sorted에 정렬된 원소를 저장할 위치 설정
11
12      while (i <= middle && j <= n) {
13          if (a[i] <= a[j]) {
14              sorted[k++] = a[i++];
15          }
16          else {
17              sorted[k++] = a[j++];
18          } // while
19
20      if (i > middle)
21          for (t = j; t <= n; t++, k++) sorted[k] = a[t];
22      else
23          for (t = i; t <= middle; t++, k++) sorted[k] = a[t];
24
25      for (t = m; t <= n; t++) a[t] = sorted[t];
26
27      printf("\n 병합 정렬 >> ");
28      for (t = 0; t < size; t++) printf("%4d ", a[t]);
29  }

```

[알고리즘 9-9] 구현

```

printf("병합전 a[%d,%d]+a[%d,%d] : ", m, middle, middle+1, n);
printf("{");
for (t = m; t <= middle; t++) printf(" %3d ", a[t]);
printf("} ");

printf("{");
for (t = middle+1; t <= n; t++) printf(" %3d ", a[t]);
printf("} ");

```



```
--
30 void mergeSort(int a[], int m, int n) {
31     int middle;
32     if (m < n) {
33         middle = (m + n) / 2;
34         mergeSort(a, m, middle);    // 앞 부분에 대한 분할 작업 수행
35         mergeSort(a, middle + 1, n); // 뒷 부분에 대한 분할 작업 수행
36         merge(a, m, middle, n);    // 부분집합에 대하여 정렬과 병합 작업 수행
37     }
38 }
```

[ex9_7.c]

```
01 #include <stdio.h>
02 void mergeSort(int a[], int m, int n);
03 int size;
04 int main(void) {
05     int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 }; // 정렬할 원소
06     size = sizeof(list) / sizeof(list[0]);           // list 배열의 원소 개수
07     printf("\n 정렬할 원소 >> ");
08     for (i = 0; i < size; i++) printf("%3d ", list[i]); // 정렬 전의 원소 출력

09     printf("\n\n<<<<<<<<<< 병합 정렬 수행 >>>>>>>>>>\n");
10     mergeSort(list, 0, size - 1); // 병합 정렬 함수 호출
11
12     getchar(); return 0;
13 }
```

7. 병합 정렬

선택 C:\WINDOWS\system32\cmd.exe

정렬할 원소 >> 69 10 30 2 16 8 31 22

<<<<<<<< 병합 정렬 수행 >>>>>>>>>>

```

병합 정렬 >> 10 69 30 2 16 8 31 22 병합전 a[0,0]+a[1,1] : { 10 } { 69 }
병합 정렬 >> 10 69 2 30 16 8 31 22 병합전 a[2,2]+a[3,3] : { 2 } { 30 }
병합 정렬 >> 2 10 30 69 16 8 31 22 병합전 a[0,1]+a[2,3] : { 2 10 } { 30 69 }
병합 정렬 >> 2 10 30 69 8 16 31 22 병합전 a[4,4]+a[5,5] : { 8 } { 16 }
병합 정렬 >> 2 10 30 69 8 16 22 31 병합전 a[6,6]+a[7,7] : { 22 } { 31 }
병합 정렬 >> 2 10 30 69 8 16 22 31 병합전 a[4,5]+a[6,7] : { 8 16 } { 22 31 }
병합 정렬 >> 2 8 10 16 22 30 31 69 병합전 a[0,3]+a[4,7] : { 2 8 10 16 } { 22 30 31 69 }

```

- [예제9-7] mergeSort() 함수를 수정하여 다음을 프로그램하시오.
 - 판매실적 자료를 저장하기 위한 구조체 struct sales 를 선언한다.
이 구조체에는 **품명과 판매실적, 제조연도** 세 개의 멤버변수가 있다.
이 구조체를 새로운 자료형 Sales 로 선언(typedef)한다.
 - main() 함수에서 Sales 자료형 배열 7개 크기를 선언하고 임의의 값으로 초기화 한다.
mergeSort_new() 를 호출하여 자료를 정렬한다.
 - mergeSort() 함수를 수정하여 mergeSort_new(Sales a[], int m, int n, int key) 함수를 작성하시오.
이 함수는 판매 자료를 오름차순으로 정렬한다.
 - 사용자로부터 key값을 입력 받고, key=1 이면 판매실적 순으로 key = 2 이면 제조연도 순으로 정렬한다.

```
Sales list[7] = {  
    {"TV", 120, 2021},  
    {"Notebook", 80, 2023},  
    {"Mouse", 150, 2020},  
    {"Keyboard", 60, 2022},  
    {"Monitor", 90, 2021},  
    {"Tablet", 110, 2024},  
    {"Phone", 200, 2022}  
};
```

8. 기수 정렬

■ 기수 정렬 radix sort의 이해

– 원소의 키값을 나타내는 기수를 이용한 정렬 방법

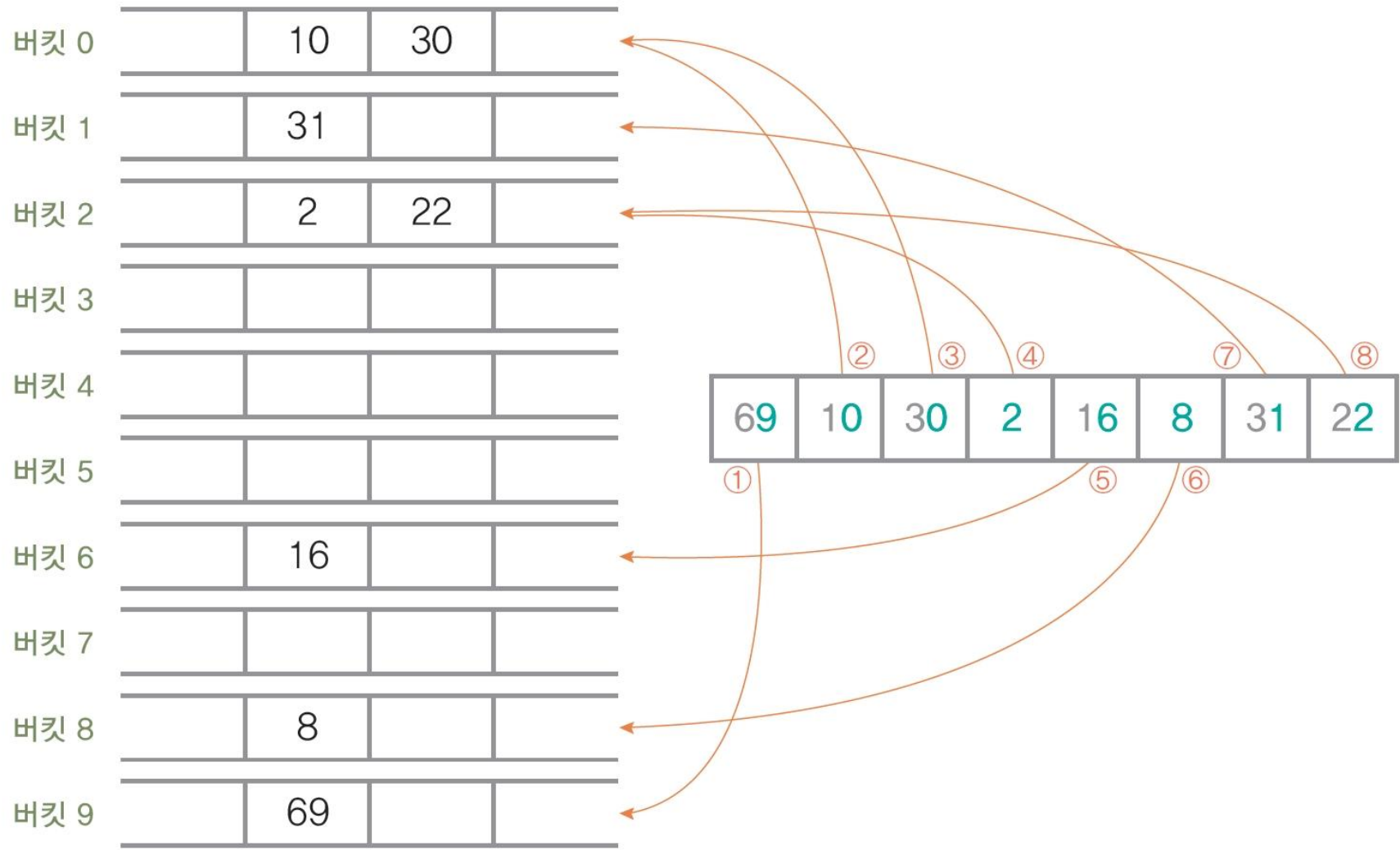
- 정렬할 원소의 키 값에 해당하는 버킷^{bucket}에 원소를 분배하였다가 버킷의 순서대로 원소를 꺼내는 방법을 반복하면서 정렬
 - 원소의 키를 표현하는 기수만큼의 버킷 사용
 - 예) 10진수로 표현된 키 값을 가진 원소들을 정렬할 때에는 0부터 9까지 10개의 버킷 사용
- 키 값의 자리수 만큼 기수 정렬을 반복
 - 키 값의 일의 자리에 대해서 기수 정렬을 수행하고,
 - 다음 단계에서는 키 값의 십의 자리에 대해서,
 - 그리고 그 다음 단계에서는 백의 자리에 대해서 기수 정렬 수행
- 한 단계가 끝날 때마다 버킷에 분배된 원소들을 버킷의 순서대로 꺼내서 다음 단계의 기수 정렬을 수행해야 하므로 큐를 사용하여 버킷을 만들

8. 기수 정렬

- {69, 10, 30, 2, 16, 8, 31, 22}의 자료를 기수 정렬 방법으로 정렬
 - 키값이 10진수이므로 0부터 9까지 열 개의 버킷을 사용하고, 키값의 최대 자릿수가 두 자리이므로 기수 정렬을 두 번 반복 수행
 - ① 키값의 1의 자리에 대해서 기수 정렬을 수행
 - (a) 정렬할 원소의 키값의 1의 자리에 맞춰 버킷에 분배
 - (b) 버킷에 분배되어 있는 원소들을 버킷 0부터 버킷 9까지 순서대로 꺼내고, 꺼낸 순서대로 저장

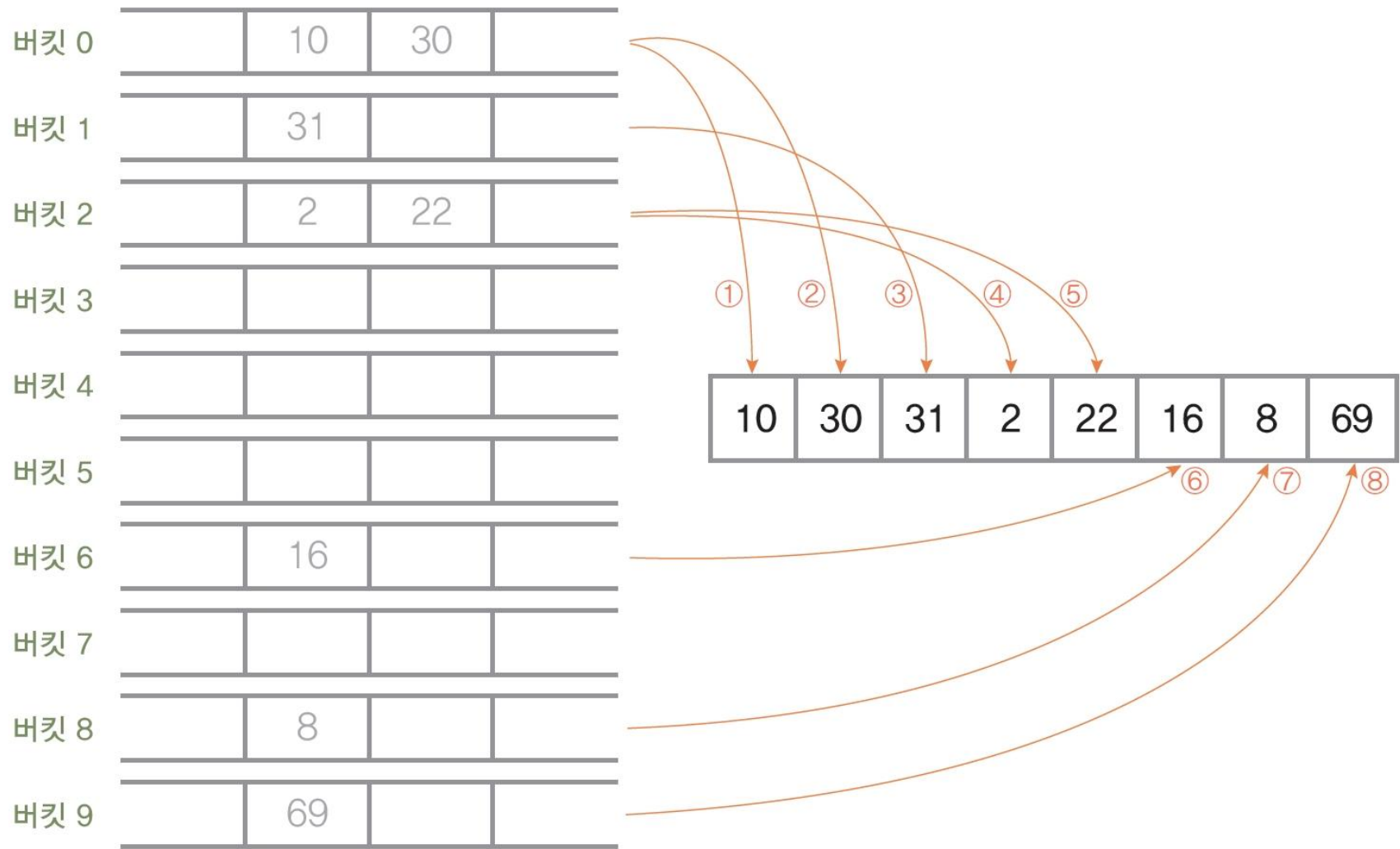
8. 기수 정렬

(a)



8. 기수 정렬

(b)

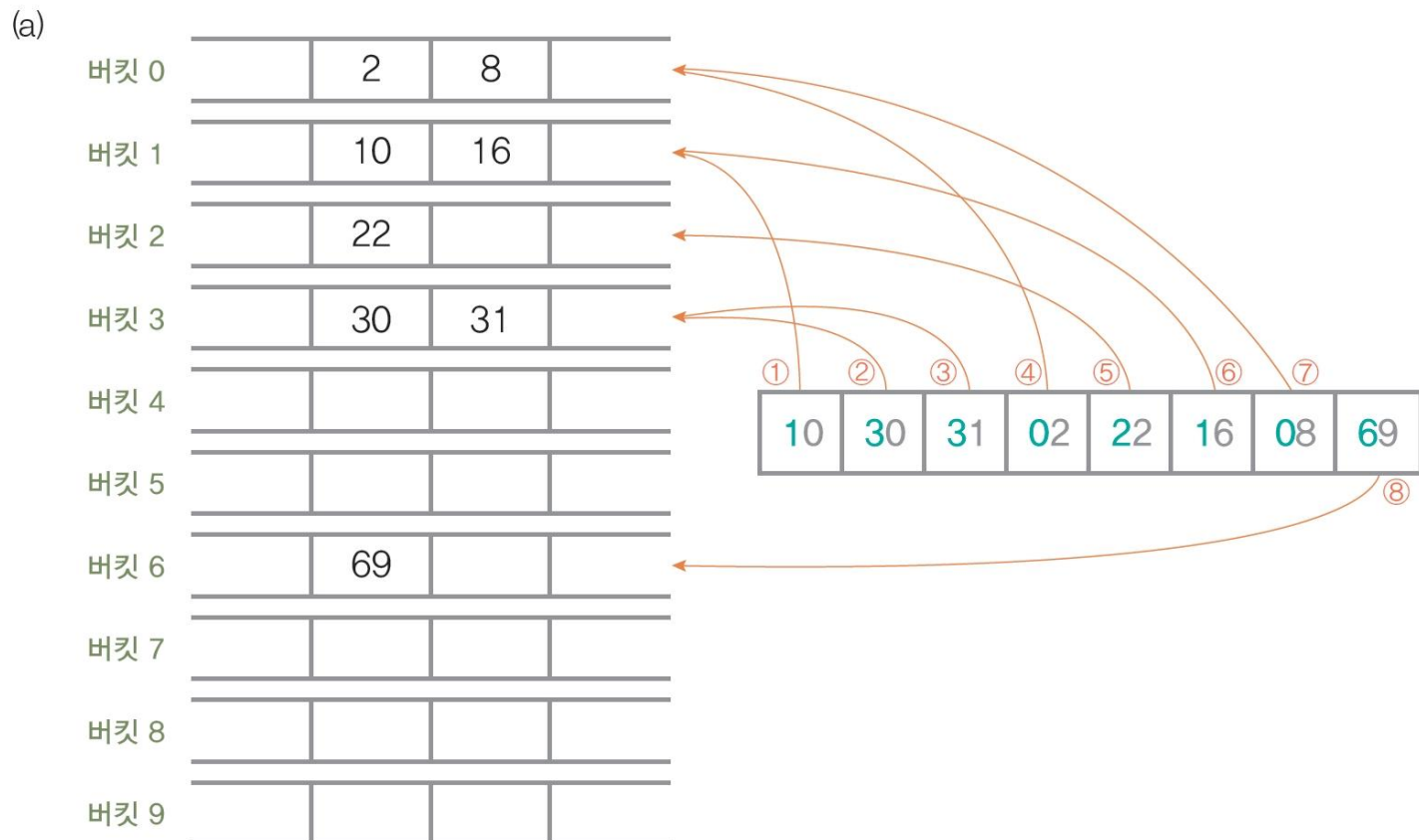


8. 기수 정렬

② 키값의 10의 자리에 대해서 기수 정렬을 수행.

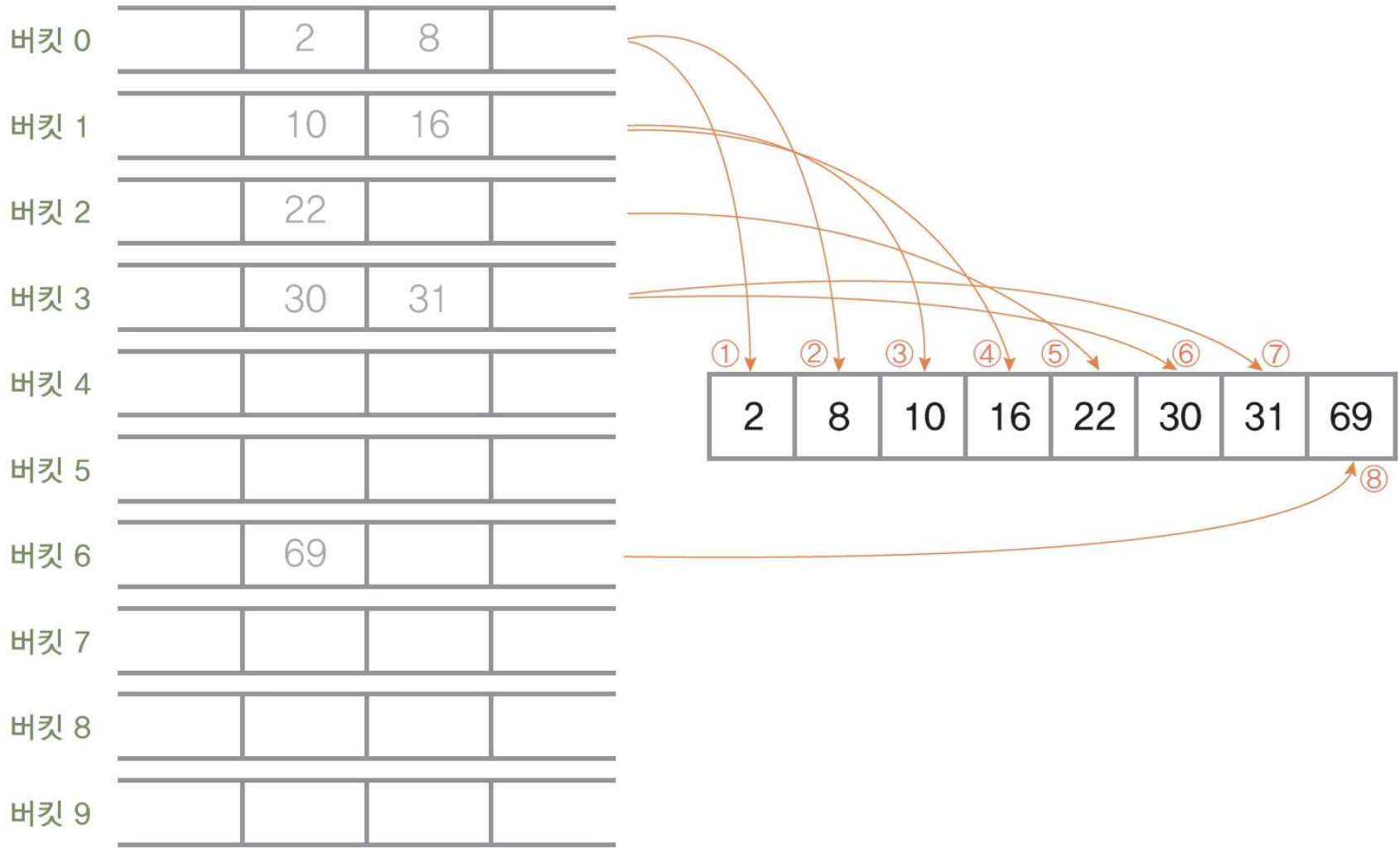
(a) 정렬할 원소의 10의 자리에 대해서 버킷에 분배.

(b) 버킷에 분배되어 있는 원소들을 버킷 0부터 버킷 9까지 순서대로 꺼내고, 꺼낸 순서대로 저장하면 전체 원소에 대한 정렬이 완성



8. 기수 정렬

(b)



8. 기수 정렬

■ 기수 정렬 알고리즘

알고리즘 9-10 기수 정렬

```

radixSort(a[], n)
  factor ← 1;
  for (k ← 0; k < d; k ← k + 1) do {           // 키값의 자릿수 d만큼 반복
    for (i ← 0; i < n; i ← i + 1) do {
      bucket ← (a[i] / factor) mod r;           // 기수 r에 대한 나머지 연산으로 버킷 주소 계산
      enqueue(Q[bucket], a[i]);                 // 버킷 큐에 원소 저장
    }
    p ← 0;
    for (i ← 0; i < r; i ← i + 1) do {
      while (Q[i] ≠ NULL) do {
        a[p] ← dequeue(Q[i]);                   // 버킷 큐 순서대로 원소를 꺼내어 저장
        p ← p + 1;
      }
    }
    factor ← factor * r;
  }
end radixSort()

```

8. 기수 정렬

- 메모리 사용 공간
 - 원소 n 개에 대해서 n 개의 메모리 공간 사용
 - 기수 r 에 따라 버킷 공간이 추가로 필요
- 연산 시간
 - 연산 시간은 정렬할 원소의 수 n 과 키 값의 자릿수 d 와 버킷의 수를 결정하는 기수 r 에 따라서 달라진다.
 - 정렬할 원소 n 개를 r 개의 버킷에 분배하는 작업 : $(n+r)$
 - 이 작업을 자릿수 d 만큼 반복
 - 수행할 전체 작업 : $d(n+r)$
 - 시간 복잡도 : $O(d(n+r))$

8. 기수 정렬

- [예제 9-8] 기수 정렬하기 : [교재 554p](#)
- 실행 결과

정렬할 원소 : 69 10 30 2 16 8 31 22

《《《《《《《《 기수 정렬 수행 》》》》》》》》》》

1 단계 : 10 30 31 2 22 16 8 69

2 단계 : 2 8 10 16 22 30 31 69

9. 힙 정렬

- 힙 정렬 heap sort의 이해
 - 7장의 힙 자료구조를 이용한 정렬 방법
 - 힙에서는 항상 가장 큰 원소가 루트 노드가 되고 삭제 연산을 수행하면 항상 루트 노드의 원소를 삭제하여 반환
 - 최대 힙에 대해서 원소의 개수만큼 삭제 연산을 수행하여 내림차순으로 정렬 수행
 - 최소 힙에 대해서 원소의 개수만큼 삭제 연산을 수행하여 오름차순으로 정렬 수행

■ 힙^{heap}의 개념

- 완전 이진 트리에 있는 노드 중에서 키값이 가장 큰 노드나 키값이 가장 작은 노드를 찾기 위해서 만든 자료구조
- 최대 힙^{max heap}
 - 키값이 가장 큰 노드를 찾기 위한 완전 이진 트리
 - {부모 노드의 키값 $\geq$ 자식 노드의 키값}
 - 루트 노드 : 키값이 가장 큰 노드
- 최소 힙^{min heap}
 - 키값이 가장 작은 노드를 찾기 위한 완전 이진 트리
 - {부모 노드의 키값 $\leq$ 자식 노드의 키값}
 - 루트 노드 : 키값이 가장 작은 노드

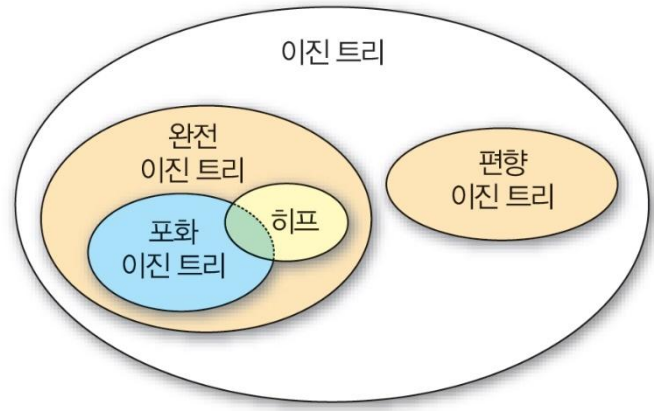
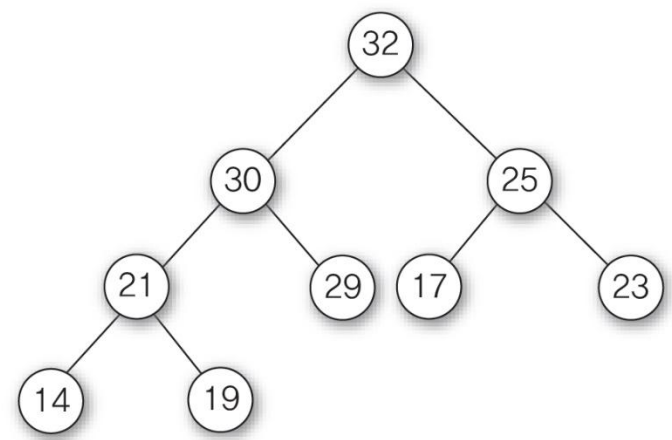
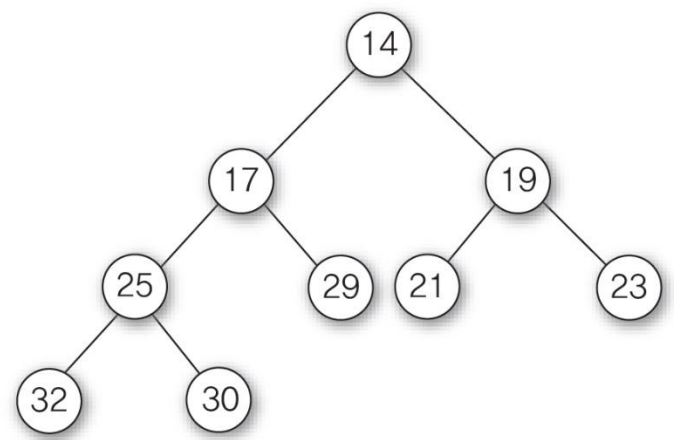


그림 7-53 이진 트리와 힙의 관계

- 힙의 예



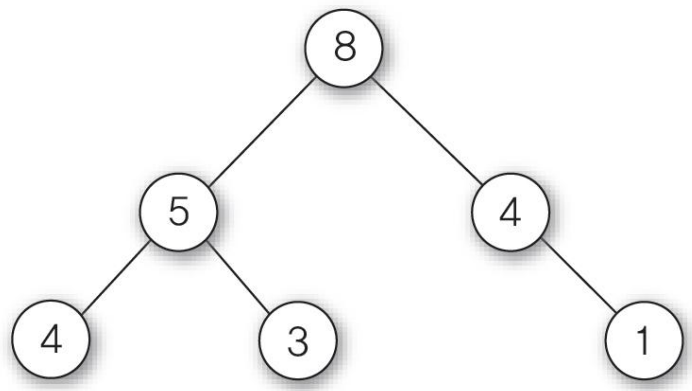
(a) 최대 힙



(b) 최소 힙

그림 7-54 힙의 예

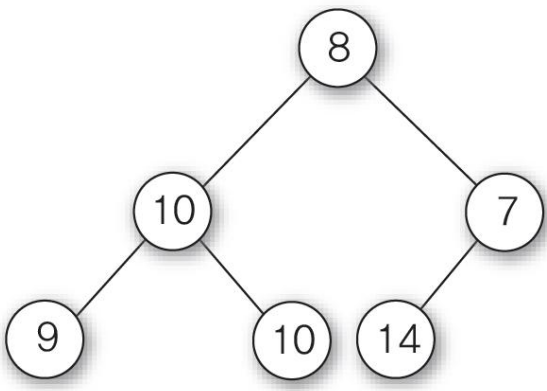
- 힙이 아닌 이진 트리의 예



(a) 완전 이진 트리가 아님

그림 7-55 힙이 아닌 이진 트리의 예

왼쪽부터 채워지지 않음



(b) 부모 노드의 키값과 자식 노드의 키값 사이의 크기 관계 미성립

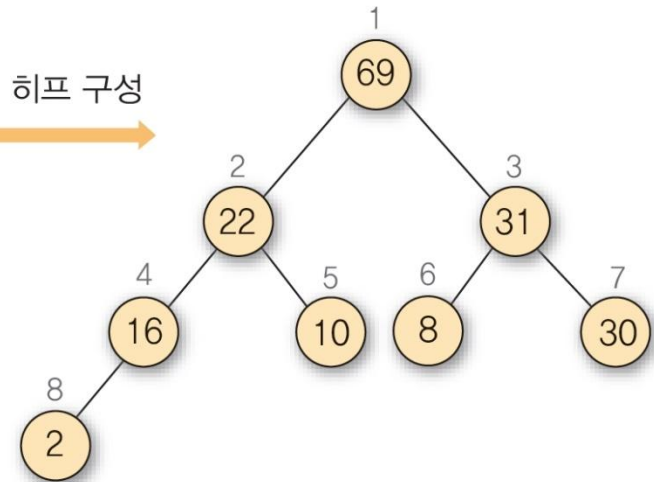
Key 값 관계 미성립

9. 힙 정렬

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 힙 정렬 방법으로 정렬하는 과정
 - 초기 상태 : 정렬할 원소에 대해 7장에서 설명한 삽입 연산을 이용해 최대 힙을 구성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
...	69	10	30	2	16	8	31	22

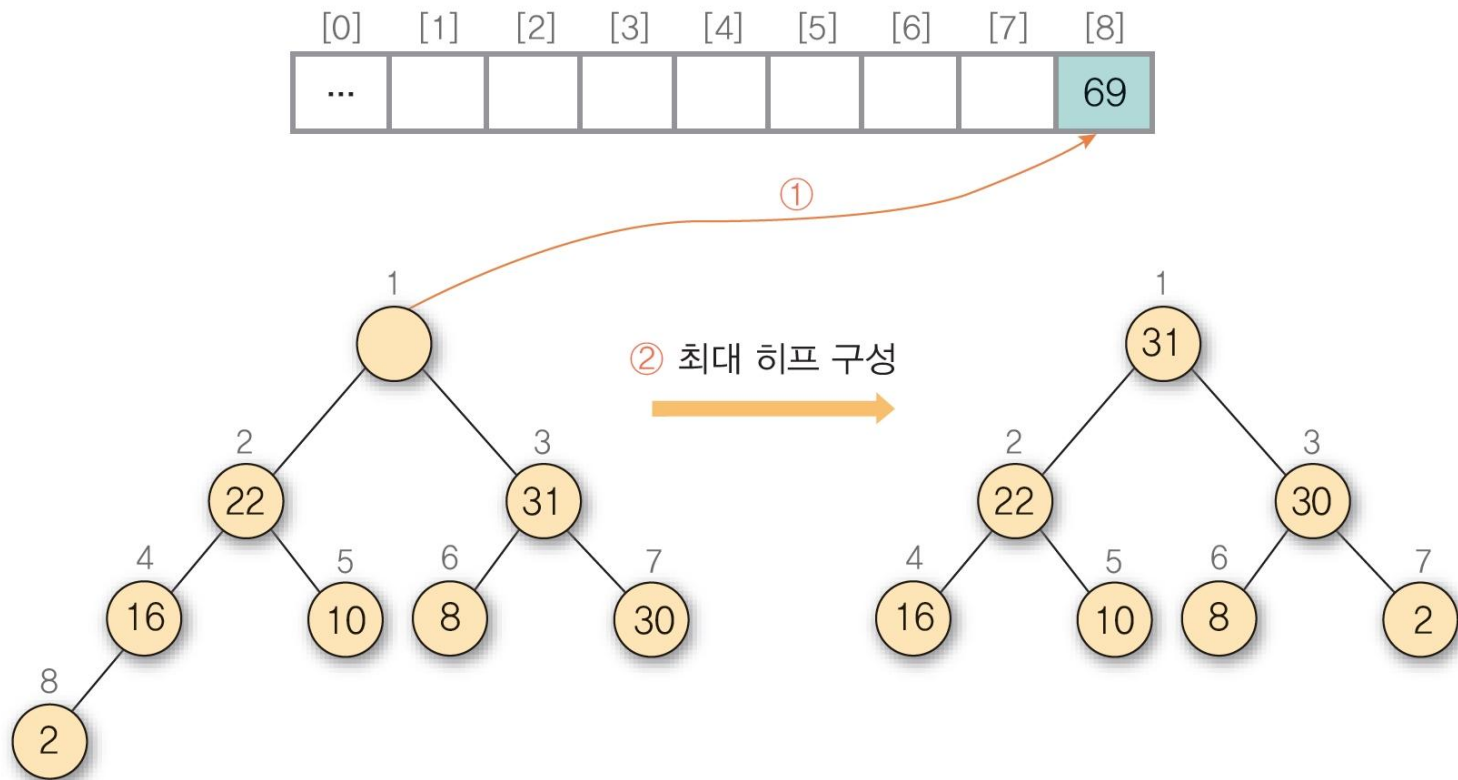
최대 힙 구성



9. 힙프 정렬

②

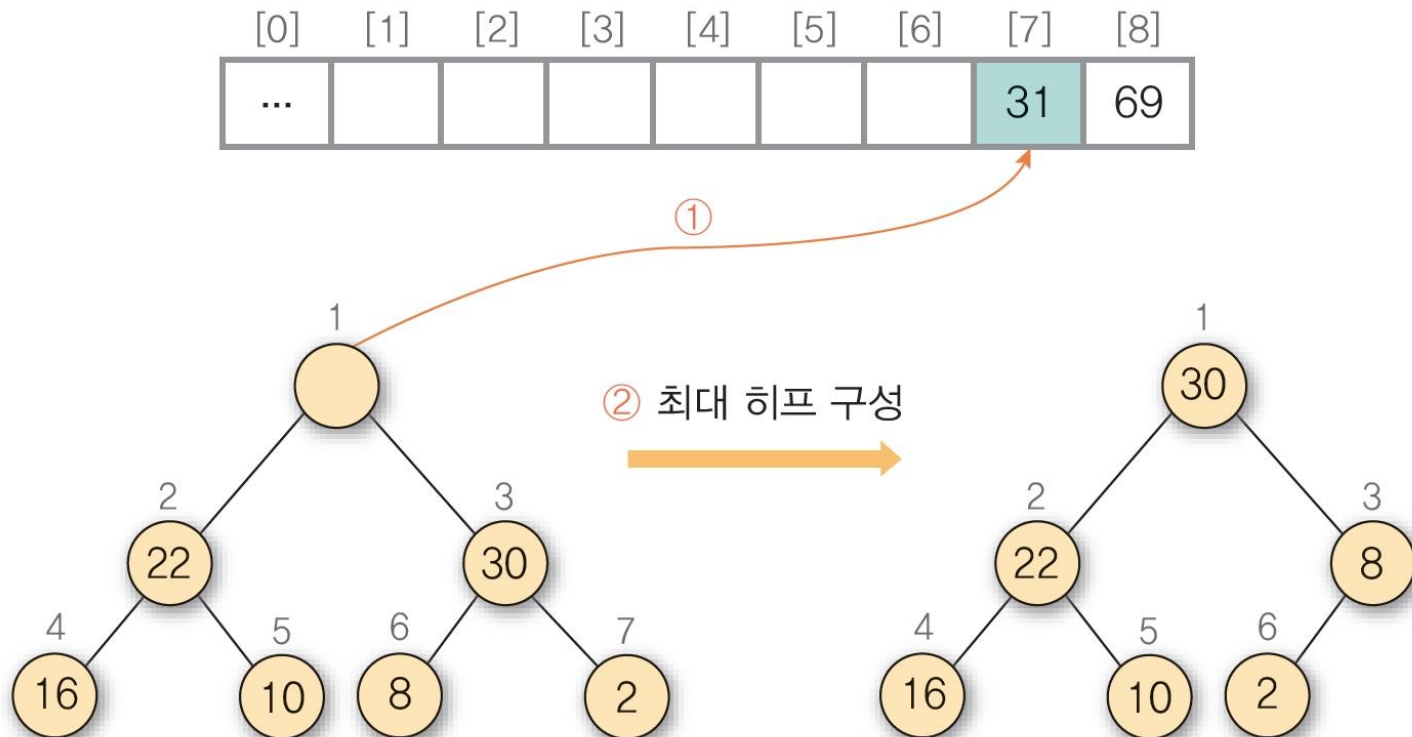
- ① 힙프에 삭제 연산을 수행하여 루트 노드의 원소 69를 구한 후 배열의 마지막 자리에 저장
- ② 나머지 힙프를 최대 힙프로 재구성. 원소를 삭제하고 힙프를 재구성하는 작업은 7장의 [알고리즘 7-12]에 따라 수행



9. 힙프 정렬

③

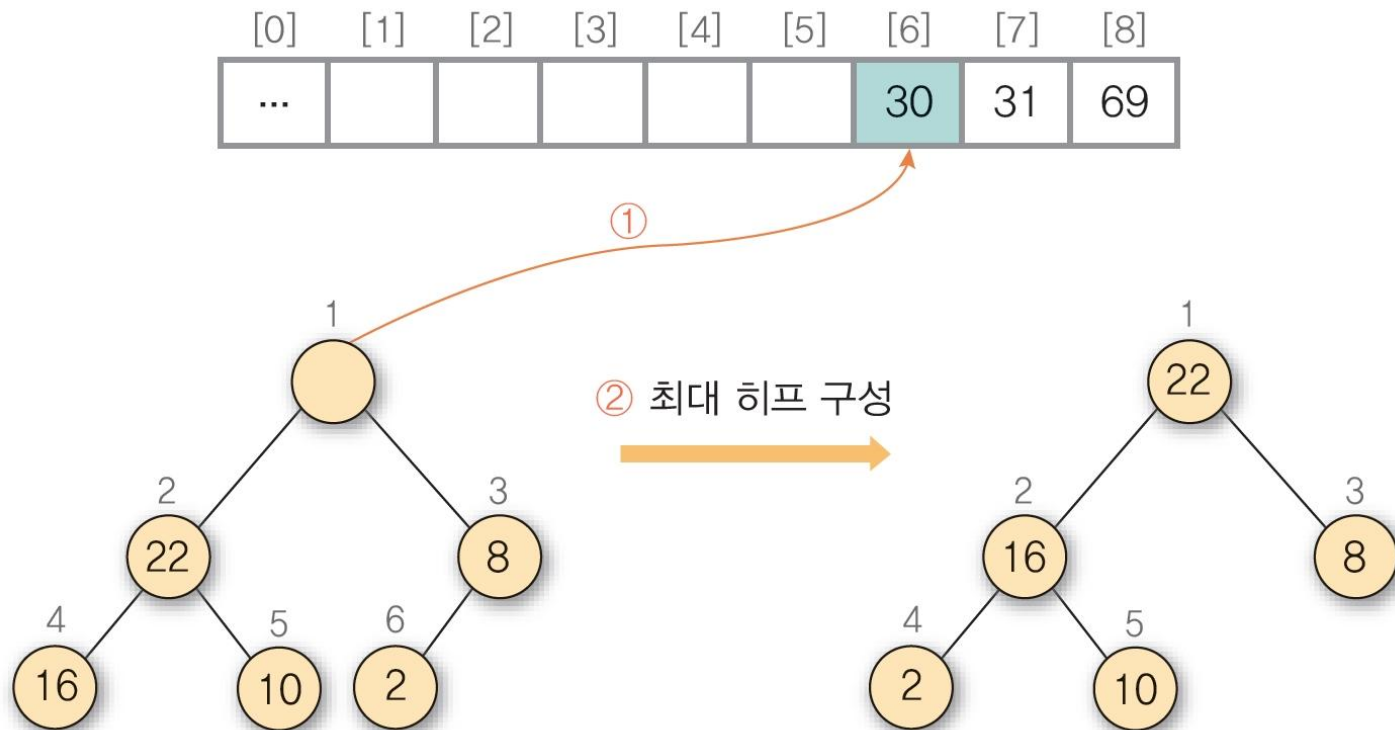
- ① 힙프에 삭제 연산을 수행하여 루트 노드의 원소 31을 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙프를 최대 힙프로 재구성



9. 힙프 정렬

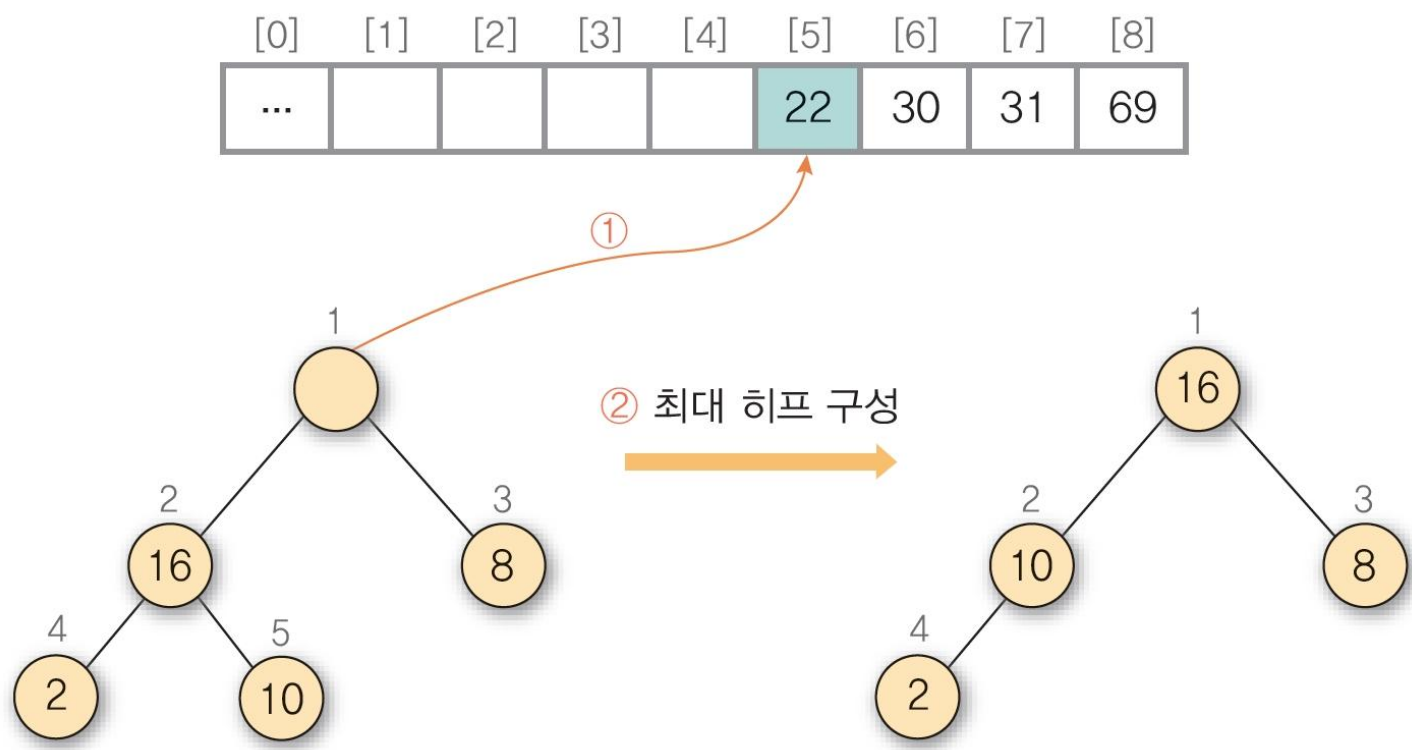
④

- ① 힙프에 삭제 연산을 수행하여 루트 노드의 원소 30을 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙프를 최대 힙프로 재구성



⑤

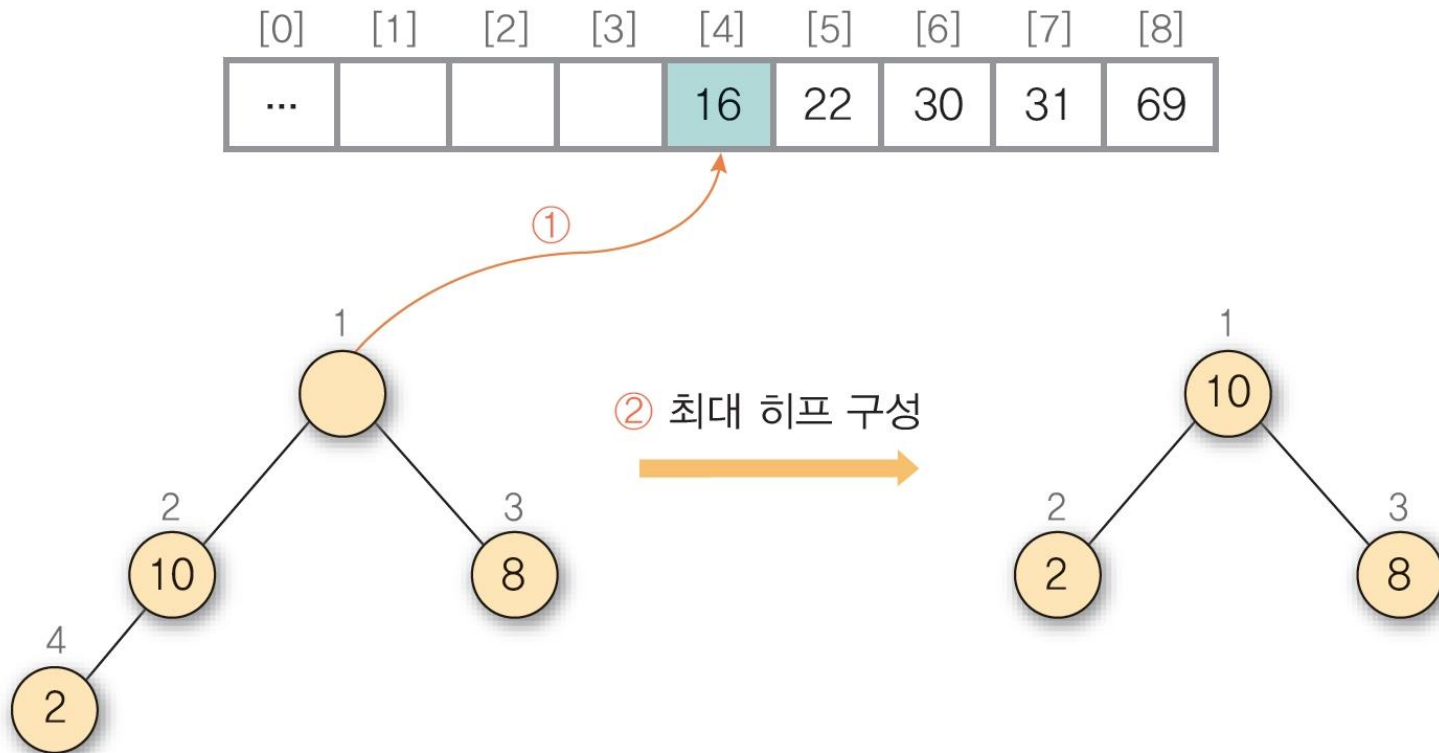
- ① 힙프에 삭제 연산을 수행하여 루트 노드의 원소 22를 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙프를 최대 힙프로 재구성



9. 힙 정렬

⑥

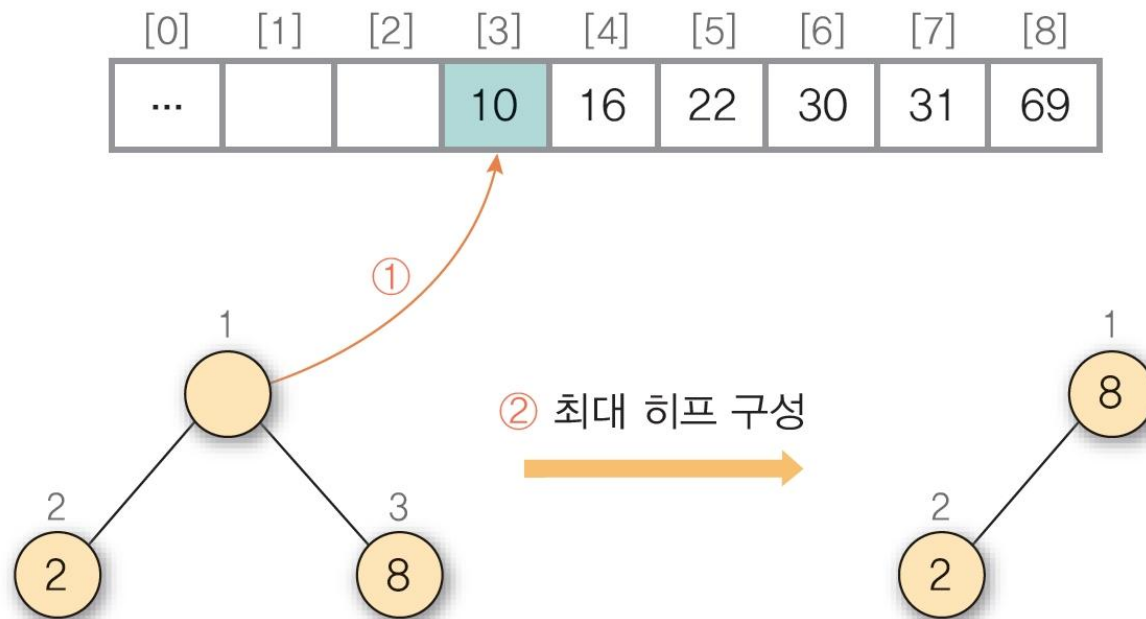
- ① 힙에 삭제 연산을 수행하여 루트 노드의 원소 16을 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙을 최대 힙으로 재구성



9. 힙프 정렬

⑦

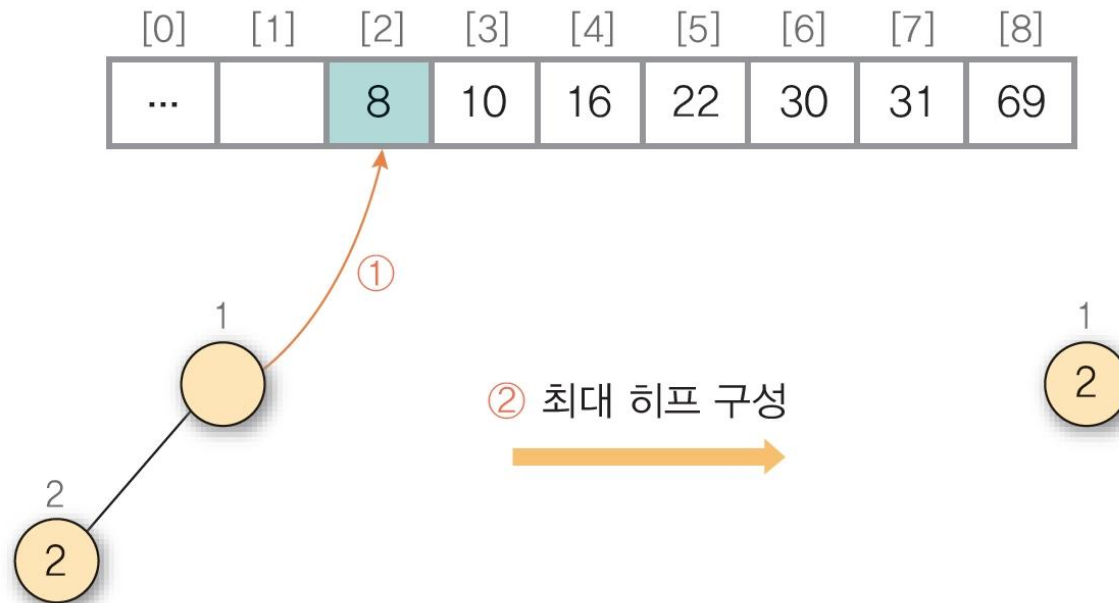
- ① 힙프에 삭제 연산을 수행하여 루트 노드의 원소 10을 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙프를 최대 힙프로 재구성



9. 힙 정렬

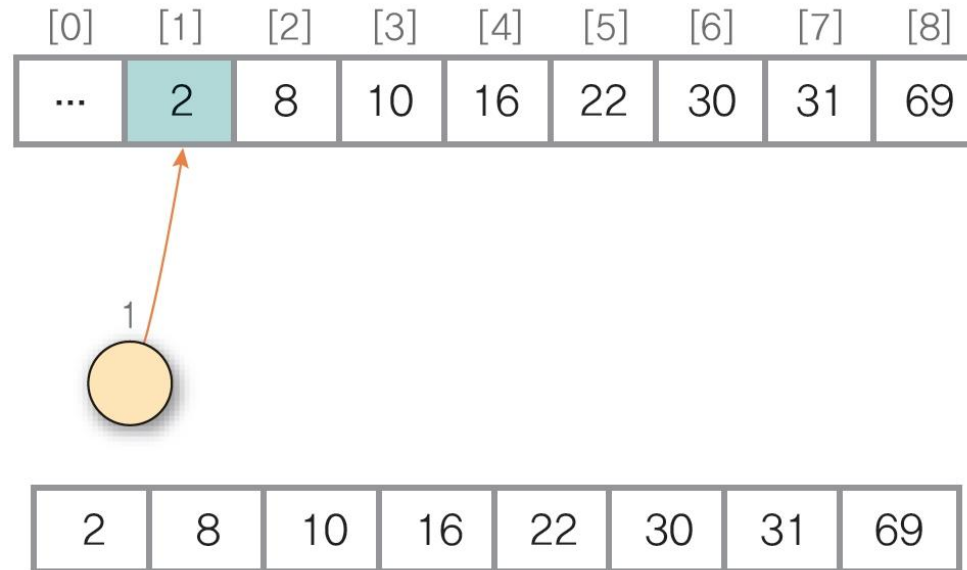
⑧

- ① 힙에 삭제 연산을 수행하여 루트 노드의 원소 8을 구한 후 배열의 비어 있는 마지막 자리에 저장
- ② 나머지 힙을 최대 힙으로 재구성



9. 힙 정렬

- ⑨ 힙에 삭제 연산을 수행하여 루트 노드의 원소 2를 구한 후 배열의 비어 있는 마지막 자리에 저장. 나머지 힙을 최대 힙으로 재구성해야 하는데 공백 힙이 되었으므로 힙 정렬을 종료



9. 힙프 정렬

■ 힙프 정렬 알고리즘

알고리즘 9-11 힙프 정렬

```

heapSort(a[])
  n ← a.length - 1;
  for (i ← n / 2; i ≥ 1; i ← i - 1) do // 배열 a[]를 힙프로 변환
    makeHeap(a, i, n);
  for (i ← n - 1; i ≥ 1; i ← i - 1) do {
    temp ← a[1];    // 힙프의 루트 노드 원소를
    a[1] ← a[i + 1]; // 배열의 비어 있는
    a[i + 1] ← temp; // 마지막 자리에 저장
    makeHeap(a, 1, i);
  }
end heapSort()

```

- 배열을 힙 구조로 재구성하는 makeHeap() 연산에 대한 알고리즘

알고리즘 9-12 힙 재구성

```
makeHeap(a[], h, m)
  for (j ← 2 * h; j ≤ m; j ← 2 * j) do {
    if (j < m) then
      if (a[j] < a[j + 1]) then j ← j + 1;
    if (a[h] ≥ a[j]) then exit;
    else a[j / 2] ← a[j];
  }
  a[j / 2] ← a[h];
end makeHeap()
```

9. 힙프 정렬

- 메모리 사용공간
 - 원소 n 개에 대해서 n 개의 메모리 공간 사용
 - 크기 n 의 힙프 저장 공간
- 연산 시간
 - 힙프 재구성 연산 시간
 - n 개의 노드에 대해서 완전 이진 트리는 $\log_2(n+1)$ 의 레벨을 가지므로 완전 이진 트리를 힙프로 구성하는 평균시간은 $O(\log_2 n)$
 - n 개의 노드에 대해서 n 번의 힙프 재구성 작업 수행
 - 평균 시간 복잡도 : $O(n \log_2 n)$

9. 힙프 정렬

- [교제 외 예제 9-9] 힙프 정렬하기
- 실행 결과

```

Microsoft Visual Studio 디버그 콘솔

정렬 전 데이터
69 10 30 2 16 8 31 22

<<<< 힙프 정렬 수행 >>>>

최대 힙프 생성
69 22 31 10 16 8 30 2
정렬 과정 : 31 22 30 10 16 8 2 69
정렬 과정 : 30 22 8 10 16 2 31 69
정렬 과정 : 22 16 8 10 2 30 31 69
정렬 과정 : 16 10 8 2 22 30 31 69
정렬 과정 : 10 2 8 16 22 30 31 69
정렬 과정 : 8 2 10 16 22 30 31 69
정렬 과정 : 2 8 10 16 22 30 31 69

정렬 후 데이터
2 8 10 16 22 30 31 69
  
```

10. 트리 정렬

- 트리 정렬^{tree sort}의 이해
 - 7장의 이진 탐색 트리를 이용하여 정렬하는 방법
 - 트리 정렬 수행 방법
 - 정렬할 원소들을 이진 탐색 트리로 구성
 - 이진 탐색 트리를 중위 우선 순회 함
 - 중위 순회 경로가 오름차순 정렬이 됨

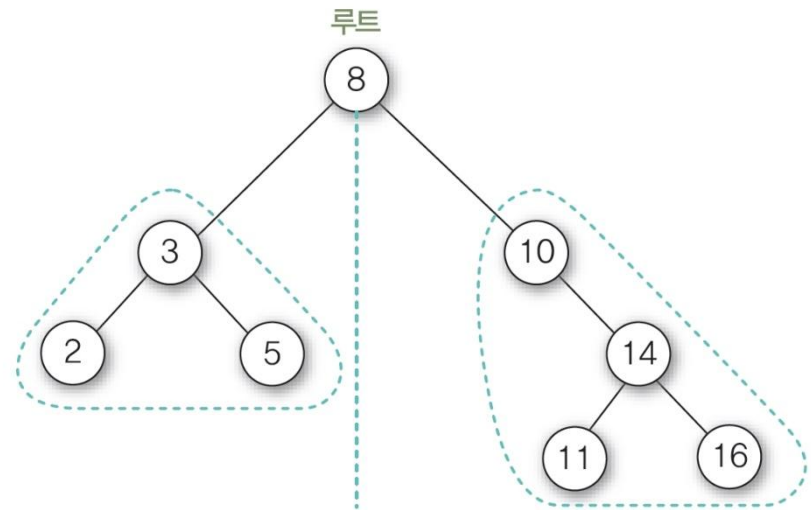
Review) 5. 이진 탐색 트리

■ 이진 탐색 트리 binary search tree

- 이진 트리를 탐색용 자료구조로 사용하기 위해 원소 크기에 따라 노드 위치를 정의한 것
- 값의 크기를 기준으로 정리해서 빠르게 찾기 위한 트리 구조
- Ex) 2, 3, 5, 8, 10, 14, 11, 16

- 모든 원소는 서로 다른 유일한 키를 갖는다.
- 왼쪽 서브 트리에 있는 원소들의 키는 그 루트의 키보다 작다.
- 오른쪽 서브 트리에 있는 원소들의 키는 그 루트의 키보다 크다.
- 왼쪽 서브 트리과 오른쪽 서브 트리도 이진 탐색 트리이다.

그림 7-32 이진 탐색 트리의 정의



왼쪽 서브 트리의 키값 < 루트의 키값 < 오른쪽 서브 트리의 키값

그림 7-33 이진 탐색 트리의 구조

Review) 5. 이진 탐색 트리

```
알고리즘 7-5 이진 탐색 트리의 노드 삽입
insertBST(bsT, x)
  p ← bsT
  while (p ≠ NULL) do {
    if (x == p.key) then return;
    q ← p;
    if (x < p.key) then p ← p.left;
    else p ← p.right;
  }

  new ← getNode();
  new.key ← x;
  new.left ← NULL;
  new.right ← NULL;

  if (bsT == NULL) then bsT ← new;
  else if (x < q.key) then q.left ← new;
  else q.right ← new;
  return;
end insertBST()
```

- 이진 탐색 트리에 원소 4를 삽입하기

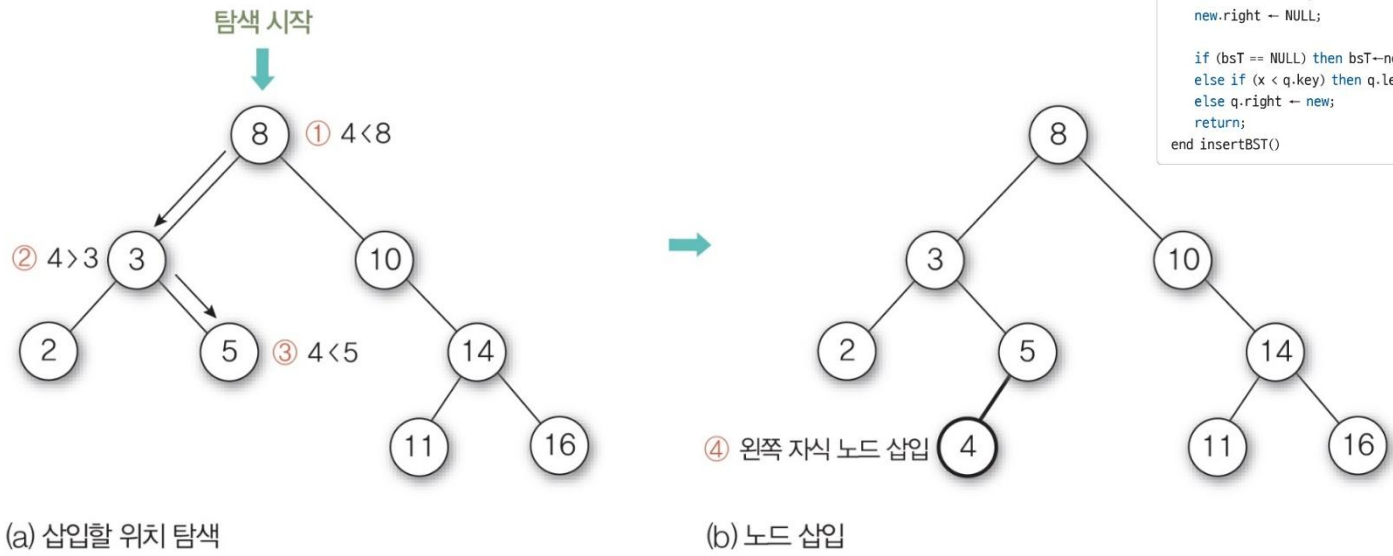


그림 7-35 이진 탐색 트리에서의 삽입 과정 예

- ① (찾는 키값 4 < 루트 노드의 키값 8)이므로 왼쪽 서브 트리를 탐색
- ② (찾는 키값 4 > 노드의 키값 3)이므로 오른쪽 서브 트리를 탐색
- ③ (찾는 키값 4 < 노드의 키값 5)이므로 왼쪽 서브 트리를 탐색해야 하지만, 왼쪽 자식 노드가 없으므로 노드 5의 왼쪽 자식 노드에서 탐색 실패가 발생
- ④ 실패가 발생한 자리, 즉 노드 5의 왼쪽 자식 노드 자리에 노드 4를 삽입

Review) 4. 이진 트리의 순회

■ 중위 순회 | inorder traversal

- $L \rightarrow D \rightarrow R$ 순서로, 현재 노드를 방문하는 작업 D를 작업 L과 작업 R의 중간에 수행

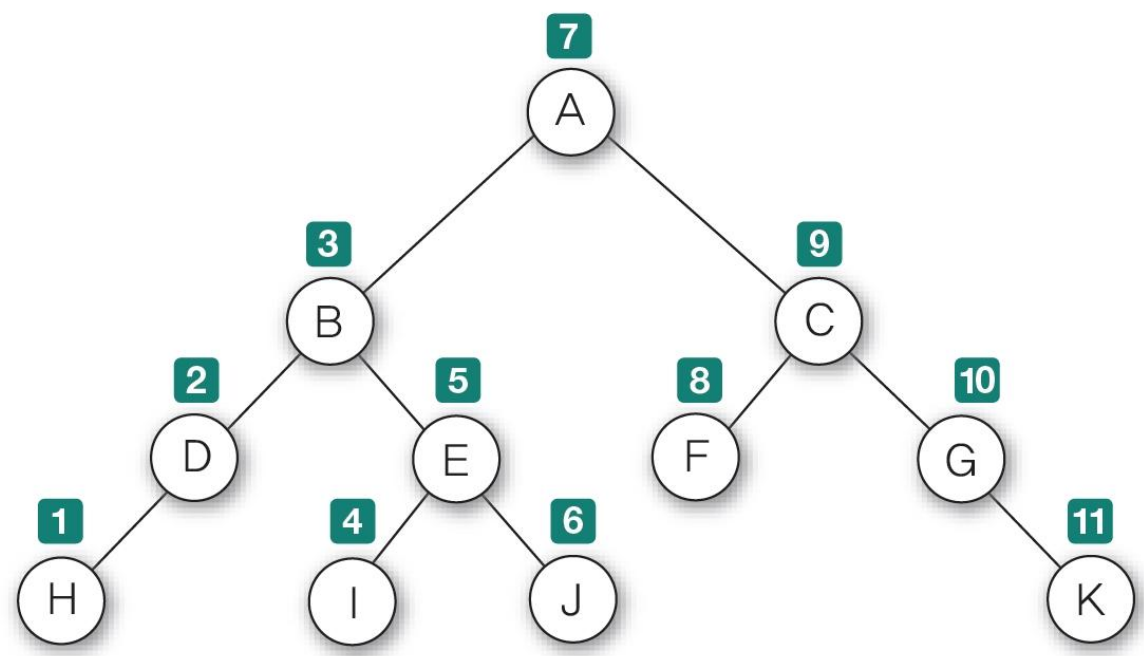
- ① 작업 L : 현재 노드 n의 왼쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)
- ② 작업 D : 현재 노드 n을 처리한다.
- ③ 작업 R : 현재 노드 n의 오른쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)

그림 7-21 이진 트리의 중위 순회 작업 순서

알고리즘 7-2 이진 트리의 중위 순회

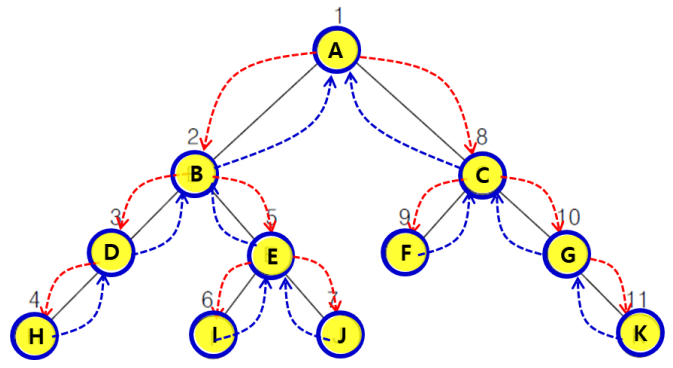
```
inorder(T)
  if (T ≠ NULL) then {
    inorder(T.left);
    visit T.data;
    inorder(T.right);
  }
end inorder()
```

- 중위 순회의 예



```
알고리즘 7-2 이진 트리의 중위 순회
inorder(T)
  if (T≠NULL) then {
    inorder(T.left);
    visit T.data;
    inorder(T.right);
  }
end inorder()
```

그림 7-22 이진 트리의 중위 순회 경로 : H-D-B-I-E-J-A-F-C-G-K



10. 트리 정렬

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 트리 정렬 방법으로 정렬하는 과정

- ① 정렬할 원소 여덟 개를 차례대로 삽입하여 이진 탐색 트리를 구성
- ② 이진 탐색 트리를 중위 순회 방법으로 순회하면서 원소를 저장



10. 트리 정렬

■ 트리 정렬 알고리즘

- 이진 탐색 트리에서의 삽입 연산 `insert( )`와 중위 순회 연산 `inorder( )`는 7장 알고리즘을 응용하여 사용함

알고리즘 9-13 트리 정렬

```
treeSort(a[], n)
  for (i ← 0; i < n; i ← i + 1) do
    insert(BST, a[i]);      // 이진 탐색 트리의 삽입 연산
    inorder(BST);           // 중위 순회 연산
end treeSort()
```

10. 트리 정렬

- 메모리 사용공간
 - 원소 n 개에 대해서 n 개의 메모리 공간 사용
 - 크기 n 의 이진 탐색 트리 저장 공간
- 연산 시간
 - 노드 한 개에 대한 이진 탐색 트리 구성 시간 : $O(\log_2 n)$
 - n 개의 노드에 대한 시간 복잡도 : $O(n \log_2 n)$

10. 트리 정렬

- [예제 9-9] 트리 정렬하기 : [교재 562p](#)
- 실행 결과

정렬할 원소 : 69 10 30 2 16 8 31 22

<<<<<<<<< 트리 정렬 수행 >>>>>>>>>>

2 8 10 16 22 30 31 69

예제 9-9 트리 정렬하기

[node.h]

```
01 // [예제 7-5]의 [node2.h] 사용
```

[node.c]

```
01 // [예제 7-5]의 [node2.c] 사용
```

[bst.h]

```
01 // [예제 7-4]의 [bst.h] 사용
```

[bst.c]

```
01 // [예제 7-4]의 [bst.c] 사용
```

[treeSort.c]

```
01 #include <stdio.h>
02 #include "bst.h"
03
```

```

04 // 트리 정렬 연산
05 void treeSort(int a[], int n) {
06     treeNode* root = NULL; // 공백 이진 탐색 트리 생성
07     root = insertBSTNode(root, a[0]); // a[0]을 공백 이진 탐색 트리의 루트로 삽입
08
09     for (int i = 1; i < n; i++) // a[1]~a[n-1]의 원소들을
10         insertBSTNode(root, a[i]); // 삽입하면서 이진 탐색 트리 구성
11     displayInorder(root); // 이진 탐색 트리를 중위 순회한 경로 출력
12 }
    
```

[알고리즘 9-13] 구현

[ex9_9.c]

```

01 #include <stdio.h>
02 void treeSort(int a[], int n);
03
04 int main(void) {
05     int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
06     int size = sizeof(list) / sizeof(list[0]); // list 배열의 원소 개수
07     printf("\n정렬할 원소 : ");
08     for (i = 0; i < size; i++) printf("%3d", list[i]);
09     printf("\n <<<<< 트리 정렬 수행 >>>>> \n\n");
10     treeSort(list, size); // 트리 정렬 함수 호출
11
12     getchar(); return 0;
13 }
    
```


- [예제9-9] treeSort() 함수를 수정하여 다음을 프로그램하시오.
 - 판매실적 자료를 저장하기 위한 구조체 struct sales 를 선언한다. 이 구조체에는 품명과 판매실적 두 개의 멤버변수가 있다. 이 구조체를 새로운 자료형 Sales 로 선언 typedef)한다.
 - main() 함수에서 Sales 자료형 배열 7개 크기를 선언하고 임의의 값으로 초기화한다. treeSort_new() 함수를 호출하여 판매자료를 판매실적의 오름차순으로 정렬하여 출력한다.
 - treeSort() 함수를 수정하여 treeSort_new() 함수를 작성하시오. 이 함수는 판매실적을 오름차순으로 정렬하여 출력 한다.
 - ex9_10ex.c 파일로 통합하여 작성하고, 이 파일과 실행 결과 캡처 화면을 함께 지정된 곳에 제출한다.

```
Sales list[7] = {  
    {"TV", 120, 2021},  
    {"Notebook", 80, 2023},  
    {"Mouse", 150, 2020},  
    {"Keyboard", 60, 2022},  
    {"Monitor", 90, 2021},  
    {"Tablet", 110, 2024},  
    {"Phone", 200, 2022}  
};
```